LAND REGISTRY, DISCOVERY & WORKFLOW

# Know the ground you stand on.

A responsive land-discovery experience connected to a live API-backed registry, with mapped parcels, public verification reports, role-specific dashboards, and transfer-review workflows.

LOCAL DEMONSTRATION | LIVE DATA CONNECTION | NOT A PRODUCTION LAND AUTHORITY

| REGISTRY PARCELS | MAP LOCATIONS | PROJECTS | API HEALTH |
|---|---|---|---|
| 5 | 3 of 5 | 2 | OK |
| Current API response | Coordinates present | Current API response | Responding on port 4001 |

EXECUTIVE OVERVIEW

## A prototype with meaningful live slices

BlockLand combines a Next.js interface, an Express/TypeScript API, PostgreSQL data accessed through Prisma, an optional Redis/BullMQ layer, and a Solidity ERC-721 registry contract. The public homepage now reads parcel data from the API and refreshes it automatically.

### CURRENT LIVE STATUS

Web: http://localhost:3200

API health: http://localhost:4001/health

Map refresh: every 60 seconds

Dataset: illustrative demo records

IMPORTANT CONTEXT

Live connection means the UI reads the running local database/API; the current records are demonstration data, not proof of legal ownership, government endorsement, or a production title service.

# Product experience

The public-facing experience is designed around discoverability first: find a parcel, understand its registry signals, inspect location and history, then take an informed next step.

## Public discovery journey

**01 SEARCH**

Search by place, plot ID, city, or landmark.

**02 EXPLORE**

Browse live parcel cards and projects.

**03 INSPECT**

Review status, location, and a public verification report.

**04 ENGAGE**

Save a parcel, request a visit, or open directions.

## What a visitor can inspect

- Registry status and parcel identifiers, including pending, verified, transferred, locked-for-sale, disputed, and under-review states.
- Map position when latitude/longitude is present; a missing coordinate is shown as unavailable rather than guessed.
- Public verification report covering owner identity signals, government verification, blockchain reference, active transfers, encumbrances, disputes, and ownership history.
- Location discovery tools: map search, basemap changes, current-location control, route display, and Street View links where external services permit.
- Project, builder, review, saved-item, dealer-contact, and site-visit interface surfaces; some are demonstration or partial-backend flows.

## Current experience notes

The homepage is a responsive HTML/JavaScript product shell mounted by the Next.js home route. Leaflet renders the map; live parcel cards are constructed from the API response. The current sample dataset has 5 parcel records, 2 projects, and 3 geolocated parcels.

A demo-data disclaimer is shown in the page footer: the records are illustrative and are not evidence of legal title or government endorsement.

Primary files: apps/web/src/app/page.tsx | apps/web/public/mark2-body.html | apps/web/public/mark2-app.js

# Integrated live map

The map is now registry-backed rather than a standalone static illustration. Its status and parcel list reflect the local API response, with refresh and coordinate handling made visible to the user.

**LIVE SNAPSHOT**

**5 parcels / 3 map pins / 2 records missing coordinates / refreshed every 60 seconds**

Observed while preparing this report; totals and statuses can change as the database changes.

## Data path

| API | > | NORMALIZE | > | MAP | > | CARDS |
|---|---|---|---|---|---|---|
| GET /api/v1/lands | | status, labels, location | | pins only for valid coordinates | | all live parcels in list |

## Map capabilities

- Live refresh: fetches the public lands endpoint at startup and repeats every 60 seconds; visible status distinguishes connected, refreshing, stale, and unavailable states.
- Live markers are rebuilt from records with numeric latitude and longitude. Current sample locations cover Hyderabad, Gurugram, and Bengaluru; two demo parcels currently have no coordinates.
- Street, satellite, and hybrid basemaps are available. Street tiles use CARTO/OpenStreetMap attribution; imagery uses Esri tiles.
- Place search uses OpenStreetMap Nominatim; driving routes use the public OSRM routing endpoint. These are external dependencies and should be replaced or governed by service agreements for production.
- Marker popups expose parcel identity/status plus route and Street View actions. Search can move the map to a matching live record.

**DATA QUALITY / LEGAL CAVEAT**

A map pin only indicates coordinates stored in the app database. It does not authenticate a survey boundary or title. Missing coordinates remain visible in the parcel list but are intentionally not placed at an invented point.

Implementation: live registry refresh and card/marker rendering in apps/web/public/mark2-app.js; map status markup in apps/web/public/mark2-body.html.

# Separate role dashboards

Role-aware pages provide distinct views for end users, government reviewers, and platform administrators. API authorization remains the source of authority; hiding or showing a page is not the security boundary.

**USER /dashboard**

Owned plots and verification status; recent transfers and escrow state; notifications; navigation to sales, purchases, escrow, and alerts.

User API: GET /api/v1/dashboard/user. Session token is read from the same browser storage used by the homepage login flow.

**GOVERNMENT /government**

Registry counts by status; verification requests; transfer approvals; open fraud cases; disputes and encumbrances. The reviewer UI provides approve/reject actions and reasons for rejections.

Government API: GET /api/v1/dashboard/government. Verification and transfer decision endpoints require GOVERNMENT or ADMIN.

**ADMIN /admin**

Platform-level land/user metrics; sanitized recent user directory; recent audit events. Dedicated route is no longer a copy of the government queue.

Admin API: GET /api/v1/admin/stats, /users, /audit-logs; these routes require ADMIN.

## Known functional boundaries

- The user sales, purchases, escrow, and alerts routes currently reuse the shared user-dashboard view; their navigation exists, but each does not yet have a dedicated filtered detail page.
- The admin console is primarily read-only today: it displays metrics, users, and audit logs; it does not expose account-role editing or comprehensive platform configuration.
- A parcel in PENDING land status is not automatically the same as a queued VerificationRequest. In the current sample, one parcel is PENDING while the officer request queue is empty; these are distinct database records.

# Architecture & data model

A workspace monorepo connects browser UI, REST services, relational business records, and a separately maintained smart-contract package.

**WEB**

Next.js 14
React + TypeScript
Leaflet + HTML/JS shell

>

**API**

Express + TypeScript
JWT / role middleware
REST + Socket.IO

>

**DATA**

PostgreSQL
Prisma ORM
Redis optional

>

**CHAIN**

Hardhat / Solidity
ERC-721 + AccessControl
separate deployment

## Core records represented

**IDENTITY & ACCESS**

User role (USER / GOVERNMENT / ADMIN), password hash, verification flags, linked wallet, refresh tokens.

**PARCEL & EVIDENCE**

Land, status, coordinates, ownership, documents, SHA-256 hashes, identity verification, verification request.

**TRANSACTION & RISK**

Transfer, escrow account, officer approval, ownership history, encumbrance, dispute, fraud flag, notification.

## Key API surfaces

- Auth and profile: /api/v1/auth; land registry and search: /api/v1/lands; projects: /api/v1/projects; transfers and escrow: /api/v1/transfers.
- Role dashboards and actions: /api/v1/dashboard; officer verification queue/actions: /api/v1/verification; public parcel report: /api/v1/verification-report/:plotId/verify.
- Additional API packages cover documents, builder discovery, site visits/reviews/engagement, fractional ownership, wallet, and admin reporting. OpenAPI/Swagger is served at /api/v1/docs.

**IMPORTANT INTEGRATION DISTINCTION**

A Solidity registry contract exists, but the reviewed transfer controller performs the demo transfer/escrow state changes in PostgreSQL, with DEMO_ESCROW references. No production chain transaction or payment-rail integration was verified. Treat blockchain hashes and escrow status as application data until end-to-end chain/payment reconciliation is implemented.

# Operational status & next steps

This is a functional local MVP/prototype, not a production government registry or legal conveyance system. The overview below separates what was observed working from what still needs production engineering.

## Verified during this overview

- Frontend responded at http://localhost:3200; API health returned {"status":"ok"} at http://localhost:4001/health.
- Live API returned five land records and two projects. Parcel statuses: 2 VERIFIED, 1 PENDING, 1 LOCKED_FOR_SALE, and 1 DISPUTED.
- Browser showed "Live registry" status, five live parcel cards, and three map markers. Missing coordinate records were labeled rather than plotted.
- Production frontend build and API TypeScript build completed successfully during the dashboard/map work; updated map script passed Node syntax validation.

## Before any real-world rollout

**01 TRUSTED DATA**

Integrate authoritative land-record sources and signed officer workflows; validate parcel coordinates/survey boundaries; define document provenance and retention.

**02 REAL SETTLEMENT**

Replace DEMO_ESCROW with an audited regulated payment/escrow provider; connect and reconcile actual on-chain transactions if blockchain is part of the legal process.

**03 SECURITY & PRIVACY**

Replace placeholder JWT secrets; establish managed secrets, HTTPS, production CORS, backups, least privilege, audit monitoring, access reviews, and PII controls.

**04 COMPLETE PRODUCT**

Build dedicated user sales/purchases/escrow/alerts pages; complete admin management workflows; add end-to-end tests for verification, rejection, cooling periods, expiry, and transfer rollback.

**05 OPERATIONS & COMPLIANCE**

Confirm jurisdictional legal requirements, accessibility, external map-provider terms/limits, incident response, support, and recovery procedures.

**LOCAL REVIEW LINKS**

Homepage / map: http://localhost:3200 | User: /dashboard | Government: /government | Admin: /admin | API docs: http://localhost:4001/api/v1/docs

Prepared from live local UI/API checks and repository source review. Demo data is illustrative; this is not legal, title, or investment advice.